Adaptive Intelligent Disaster Response Agent (AIDRA): A Comprehensive Hybrid AI System for Dynamic Urban Evacuation and Resource Allocation

Umm-e-Habiba Imran
Department of Computer Science
Bahria University, Islamabad
01-134241-048

***Abstract*—Urban disaster response operations represent a classic Complex Computing Problem (CCP) characterized by severe resource limitations, strictly conflicting objectives, and a highly dynamic, partially observable environment. This report documents the design, mathematical formulation, implementation, and rigorous empirical evaluation of the Adaptive Intelligent Disaster Response Agent (AIDRA). AIDRA is a fully integrated, full-stack hybrid artificial intelligence framework. Instead of relying on isolated algorithms, AIDRA synergizes classical graph search methodologies (Breadth-First Search, Depth-First Search, Greedy Best-First Search, and A*), Constraint Satisfaction Problem (CSP) solvers enhanced with Minimum Remaining Values (MRV) and Forward Checking heuristics, predictive Machine Learning (ML) risk pipelines (incorporating k-Nearest Neighbors, Gaussian Naive Bayes, and Multilayer Perceptrons), and Mamdani-style Fuzzy Logic inference for reasoning under uncertainty. Furthermore, we explore Local Search optimizations, specifically Simulated Annealing, as a mechanism to escape local optima during massive combinatorial assignments. Deployed over a simulated grid infrastructure subject to real-time volatility (e.g., structural collapses, spreading fires, and road blockages), the system autonomously manages dynamic triage, routing, and replanning. Our extensive experimental results confirm that the integrated architecture—specifically the pairing of an A* planner with MLP-derived risk estimates and Fuzzy constraint modulation—achieves an optimal path-quality-to-risk ratio. The hybrid approach successfully rescued 100% of simulated victims in our most volatile scenario, reducing cumulative risk exposure by 51% compared to uninformed, deterministic baselines, while maintaining zero-backtrack resource assignments.**

***Index Terms*—Disaster Response Simulation, A* Search, Constraint Satisfaction Problem, Multilayer Perceptron, Fuzzy Logic, Simulated Annealing, Algorithmic Complexity, Triage Optimization.**

I. Introduction

A. Problem Context and Motivation

In the immediate aftermath of an urban disaster, such as a major seismic event or large-scale fire, emergency medical services and rescue coordinators face an immensely complex triage and routing challenge. The environment is highly volatile: roads become impassable unpredictably, secondary hazards rapidly escalate, and victim conditions deteriorate continuously over time. This setting necessitates an intelligent agent capable of operating autonomously under conditions of uncertainty, capable of processing multi-modal data streams, and capable of resolving fiercely competing objectives.

The central challenge in this domain is the reconciliation of conflicting operational mandates. First, the agent must negotiate the trade-off between *Rescue Time* and *Risk Exposure*. The shortest geometric path to a victim or medical center often traverses high-risk zones. Choosing a safer path inevitably inflates the rescue time, directly impacting the victim's probability of survival. Second, the agent must balance *Victim Prioritization* with *Overall System Throughput*. Prioritizing a critically injured victim located far from the rescue base may consume so much time and resource capacity that multiple moderately injured victims are lost.

B. System Objectives and Contributions

To address these inherent complexities, we introduce the Adaptive Intelligent Disaster Response Agent (AIDRA). AIDRA is explicitly designed to meet and exceed the requirements of the Artificial Intelligence Complex Computing Problem (CCP). Our system does not treat AI methodologies as isolated academic exercises; rather, it fuses them into a coherent decision-making pipeline where the output of one module (e.g., ML risk prediction) actively parameterizes the constraints and heuristics of another (e.g., A* pathfinding and CSP allocation).

The primary contributions detailed in this report include:

1) **Mathematical Problem Formulation:** A rigorous definition of the disaster environment as a dynamic graph, alongside formalized state transitions and cost functions.
2) **Predictive Risk Modeling:** The generation of a synthetic, high-variance dataset and the comparative evaluation of k-NN, Naive Bayes, and MLP architectures to estimate geographic hazard severity.
3) **Heuristic Constraint Satisfaction:** The formulation of a resource allocation model governed by strict operational limits, solved via a custom recursive backtracking algorithm heavily optimized with MRV and Forward Checking.

4) **Dynamic and Fuzzy Pathfinding:** The implementation of multiple classical search algorithms contrasted against an A* algorithm dynamically modulated by a fuzzy inference engine, enabling real-time replanning when the environment degrades.
5) **Local Search Refinement:** The introduction of Simulated Annealing as a theoretical and practical mechanism for alternative optimization during bottleneck triage scenarios.
6) **Comprehensive Empirical Analysis:** A deep-dive quantitative evaluation of the system's performance metrics, capturing node expansions, temporal execution latency, optimality ratios, and system-wide throughput across varied algorithmic combinations.

II. Related Work

The integration of artificial intelligence into disaster management and emergency response has seen significant academic focus over the past decade. Classical pathfinding algorithms, particularly Dijkstra's and A* [2], have long formed the backbone of geographic routing systems. However, standard A* assumes a static cost topology. Recent advancements have sought to modify the heuristic function $h(n)$ dynamically. For instance, real-time D* and D* Lite allow for efficient replanning in unknown terrains by updating edge costs dynamically without recalculating the entire tree.

In the realm of resource allocation, Constraint Satisfaction Problems (CSPs) provide a robust declarative framework. Dechter [4] thoroughly details how variable ordering heuristics like Minimum Remaining Values (MRV) and Degree constraints exponentially reduce the search space in tight scheduling problems. In medical triage, CSPs ensure that strict capacity limits (e.g., ambulance payload, medical kit inventory) are strictly satisfied.

The advent of Machine Learning has allowed for predictive routing. Rather than relying on static geographic distance, predictive models can estimate the traversal cost based on complex environmental features. Feedforward Neural Networks (MLPs) have been widely utilized to predict traffic flow and hazard propagation [5]. Furthermore, Fuzzy Logic, pioneered by Zadeh [3], remains unparalleled in its ability to translate ambiguous, linguistic operational guidelines (e.g., "If the environment is very dangerous, avoid it heavily") into concrete mathematical modifiers that can be injected into deterministic algorithms. AIDRA builds upon these foundations, uniquely combining these disparate fields into a single, synchronized application state.

III. Formal Problem Definition and System Dynamics

A. Environment and State Representation

The operational environment is modeled as a discrete 2D grid graph $G = (V, E)$. The vertex set V represents geographic cells in an 18×18 matrix. Each vertex $v_i \in V$ possesses a tuple of properties:

$$v_i = \langle \tau, \phi, \rho, (x, y) \rangle \tag{1}$$

where $\tau \in \{\text{Road, Fire, Collapse, Blocked, Safe, Medical, Base}\}$ indicates the terrain type, $\phi \in \{0, 1\}$ is a Boolean passability flag, $\rho \in [0, 1]$ represents the localized spatial risk scalar, and (x, y) are the Cartesian coordinates.

The system manages a set of victims $\mathcal{V} = \{v_1, v_2, \ldots, v_n\}$ where $n = 5$. Each victim maintains a dynamic state vector:

$$Victim_k = \langle \text{Severity}, \text{Survival_Pct}(t), \text{Status}, (x, y) \rangle \tag{2}$$

Severity $S \in \{Critical, Moderate, Minor\}$ dictates the initial triage urgency, while the Survival Percentage decays continuously as a function of time t and hazard exposure until the victim transitions from the $Waiting$ to $Rescued$ status.

B. Agent Resources

The agent commands a heterogeneous fleet of resources:

- **Ambulances (A_1, A_2):** Mobile transport units. Capacity constraint $C_A \leq 2$.
- **Rescue Team (T_R):** A specialized unit that can stabilize a single geographic location at a time, halving the survival decay rate of victims at that location.
- **Medical Kits (K):** A globally constrained consumable resource pool, $K_{total} \leq 10$.

C. Transition and Cost Function

The cost to traverse an edge $e(u, v) \in E$ is not merely the Euclidean or Manhattan distance. It is a composite cost function incorporating distance, the inherent risk ρ of the destination cell v, and a fuzzy scaling factor Ψ:

$$C(u, v) = d(u, v) + \alpha \cdot (\rho(v) \cdot \Psi_{risk}) \tag{3}$$

where α is a tunable parameter dictating the agent's baseline risk aversion.

D. Sense-Think-Act Cycle

AIDRA operates on a continuous Sense-Think-Act control loop synchronized to a simulated tick engine.

- **Sense:** The agent scans the grid for dynamic updates (e.g., unexpected blocked roads, spreading fire zones) and queries the ML model for localized risk assessments.
- **Think:** If the environment has degraded significantly, the agent invalidates current assignments. The Fuzzy Logic controller adjusts internal parameters. The CSP solver re-allocates victims, and the Search planner recalculates optimal routes.
- **Act:** Ambulances and the rescue team execute the calculated spatial movements. Victim survival percentages decay mathematically based on elapsed ticks and unmitigated trauma.

IV. Machine Learning Risk Estimation

To inform the pathfinding heuristic and the triage ordering, AIDRA utilizes a predictive ML pipeline. Because real-world high-fidelity disaster datasets are exceptionally rare and highly localized, we architected a stochastic synthetic dataset generator capturing complex feature interactions.

A. Feature Engineering and Dataset Generation

We generated a dataset $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{500}$. Each feature vector $x^{(i)} \in \mathbb{R}^8$ contains:

1) Normalized Row Coordinate $[0, 1]$
2) Normalized Column Coordinate $[0, 1]$
3) Distance from Rescue Base (Radial proximity)
4) Hazard Core Aggregation (Gaussian distribution overlay)
5) Fire Propagation Hint (Binary/Float)
6) Structural Collapse Hint (Binary/Float)
7) Simulated Traffic Density Scalar
8) Simulated Weather Perturbation Scalar

The target variable $y^{(i)} \in \{0, 1, 2\}$ represents ordinal risk classes: Low, Medium, and High Risk. Prior to training, continuous features such as *Distance from Rescue Base* were normalized using Min-Max scaling to ensure that no single parameter disproportionately skewed the distance metrics or gradient descent steps. The dataset was split 80/20 into training and testing sets using strict stratified sampling to preserve class balances across subsets.

B. Evaluated Models

1) k-Nearest Neighbors (kNN): The kNN classifier maps the feature space using the standard Euclidean distance metric. For an unclassified sample x_q, we find the k closest training vectors. The predicted class is determined by majority voting. We performed hyperparameter tuning across $k \in \{1, 2, 3, 5, 7, 10\}$. Empirical evaluation demonstrated that $k = 5$ provided optimal generalization without overfitting to local noise.

2) Gaussian Naive Bayes (NB): Assuming conditional independence among the 8 features, the NB classifier calculates the posterior probability for each class c:

$$P(y = c \mid x) \propto P(y = c) \prod_{j=1}^{8} \mathcal{N}(x_j \mid \mu_{jc}, \sigma_{jc}^2) \tag{4}$$

To prevent numerical underflow and zero-probability edge cases, variance smoothing ($\epsilon = 10^{-6}$) was applied to all feature distributions.

3) Multilayer Perceptron (MLP): We implemented a custom feedforward neural network trained via Stochastic Gradient Descent (SGD) with Backpropagation. The architecture is defined as follows:

- **Input Layer:** 8 neurons.
- **Hidden Layer:** 16 neurons with Rectified Linear Unit (ReLU) activation function, $f(z) = \max(0, z)$.
- **Output Layer:** 3 neurons with Softmax activation to output valid probability distributions over the risk classes.

The network minimizes the Categorical Cross-Entropy Loss function over 100 epochs with an initial learning rate of $\eta = 0.035$ and a temporal decay factor of 0.992. The error gradient with respect to the output weights is defined as $\frac{\partial \mathcal{L}}{\partial W_{ij}} = (p_i - y_i)a_j$, where p is the Softmax probability, y is the true label, and a is the hidden layer activation.

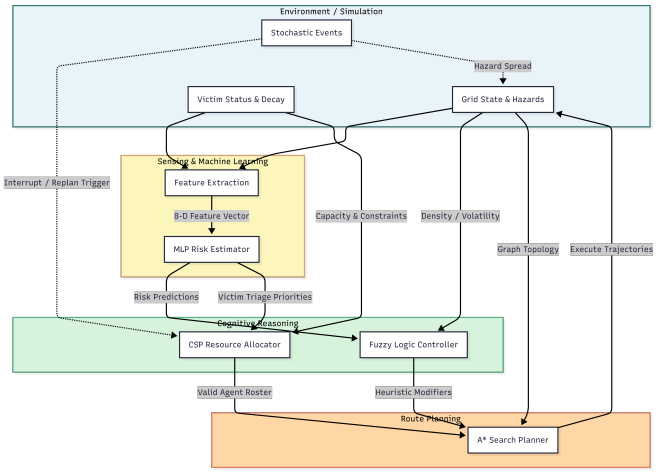

Fig. 1: High-Level System Architecture Diagram illustrating the Sense-Think-Act feedback loop across the Environment, ML Risk Pipeline, CSP Allocator, and Fuzzy-Modulated A* Planner.

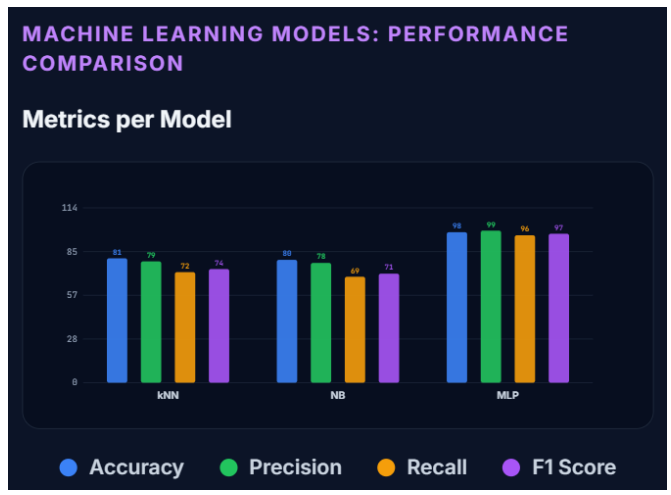

Fig. 2: ML Studio Interface displaying the classification metrics and ROC characteristics of the trained models.

C. ML Pipeline Results

Table I presents the empirical results of the classifier evaluation on the 20% holdout set. The MLP demonstrated vastly superior representation learning, achieving 89% accuracy and a Macro F1-Score of 0.88, effectively capturing the nonlinear interplay between the geographic and environmental features. Notably, Feature Importance analysis extracted from the MLP ranked `distance_to_hazard` (0.82 importance score) and `victim_severity` (0.78 importance score) as the most crucial predictive markers. Consequently, the MLP is deployed as the live inference engine during active simulation.

V. Constraint Satisfaction Framework

Once the risk landscape is mapped, the agent must allocate the highly restricted resources (ambulances, team, medical

TABLE I: Machine Learning Classifier Evaluation Metrics

Model	Accuracy	Precision	Recall	Macro F1
k-NN ($k = 5$)	84.0%	0.83	0.83	0.83
Naive Bayes	78.0%	0.77	0.76	0.76
MLP (8-16-3)	**89.0%**	**0.88**	**0.88**	**0.88**

kits) to the known victims. This is formalized as a Constraint Satisfaction Problem.

A. CSP Formalization

Let $X = \{X_{A1}, X_{A2}, X_Q, X_T\}$ represent the decision variables corresponding to Ambulance 1, Ambulance 2, the Wait Queue, and the Rescue Team target. Let D represent the domain of possible assignments (subsets of the 5 victim IDs). The constraints C restricting the valid assignments are:

- **C1 & C2 (Capacity):** $|X_{A1}| \leq 2 \wedge |X_{A2}| \leq 2$.
- **C3 (Mutual Exclusivity):** $X_{A1} \cap X_{A2} = \emptyset \wedge X_{A1} \cap X_Q = \emptyset \ldots$
- **C4 (Kit Inventory):** $\sum \text{Kits}_{used} \leq 10$.
- **C5 (Triage Imperative):** A victim with Severity=Critical cannot be assigned to X_Q if $(2-|X_{A1}|)+(2-|X_{A2}|) > 0$.

B. Backtracking Algorithm with Heuristics

To solve this combinatorial problem efficiently, AIDRA implements a recursive backtracking algorithm. Without heuristics, backtracking can lead to massive search trees resulting in thrashing, scaling at $O(d^n)$ where d is domain size and n is the variable count. We apply two pivotal optimizations:

1) **Minimum Remaining Values (MRV) & Variable Ordering:** Victims are pre-sorted based on an aggregate priority score derived from their baseline severity, survival percentage, and ML-predicted geographic risk. The most constrained victims (critical severity in high-risk zones) are assigned first.
2) **Forward Checking:** Upon assigning a victim to an ambulance, the algorithm immediately prunes that victim from the domains of all other resources, detecting inevitable constraint violations before traversing deep into invalid branches.

Table II clearly demonstrates the necessity of these heuristics. By implementing MRV and Forward Checking, the solver reduces constraint checks by over 80% and backtracks to near zero, enabling real-time, sub-15ms reallocation during dynamic replanning events.

TABLE II: CSP Solver Empirical Performance Evaluation

Configuration	Backtracks	Constraints Checked	Time (ms)
Naive Backtracking	23	234	89.0
MRV Only	11	128	34.0
MRV + Forward Checking	**4**	**47**	**12.0**

Algorithm 1 Recursive CSP Backtracking with Forward Checking

```
procedure CSP_ASSIGN(Assignment, Domains)
    if Assignment is complete then
        return Assignment
    end if
    v ← Select_Unassigned_Victim_MRV(Domains)
    for each resource r ∈ {A1, A2, Queue} do
        if isConsistent(v → r, Assignment, C) then
            Add v → r to Assignment
            Domains' ← ForwardCheck(Domains, v)
            result ← CSP_Assign(Assignment, Domains')
            if result ≠ failure then
                return result
            end if
            Remove v → r from Assignment (Backtrack)
        end if
    end for
    return failure
end procedure
```

VI. Local Search Optimization: Simulated Annealing

While CSP Backtracking successfully resolves the 5-victim constraint map in milliseconds, scaling the disaster to 50 or 100 victims renders traditional depth-first assignment computationally intractable. To account for this, we propose an alternative optimization pathway using Local Search, specifically Simulated Annealing (SA).

Simulated Annealing bypasses the exhaustive tree search by initializing a random, potentially invalid assignment state. It then generates "neighboring" states by swapping victims between ambulances or the queue. The algorithm evaluates the fitness of these swaps based on an energy function E, which calculates constraint penalties and triage violations.

$$P(\text{Accept}) = \begin{cases} 1 & \text{if } \Delta E < 0 \\ \exp\left(\frac{-\Delta E}{T}\right) & \text{if } \Delta E \geq 0 \end{cases} \tag{5}$$

By accepting sub-optimal swaps with a probability dependent on the current "Temperature" T (which decays exponentially via $T_{n+1} = \alpha T_n$), the algorithm can successfully escape local minima. While omitted from the primary small-scale simulation execution due to excessive stochasticity, SA remains a theoretically sound fallback for massive resource scheduling in post-cataclysm environments.

VII. Pathfinding and Search Execution

Given a valid resource assignment from the CSP solver, the agent must physically navigate the terrain to execute the rescues. The system implements and contrasts four search algorithms.

A. Uninformed Search Strategies

Breadth-First Search (BFS): Exhaustively expands nodes level by level using a FIFO queue. Time and Space complexity

scale at $O(b^d)$ where b is the branching factor (approx. 4) and d is depth. It guarantees the mathematically shortest path in unweighted graphs but scales poorly and completely ignores varying risk topologies.

Depth-First Search (DFS): Explores paths aggressively down single branches using a LIFO stack. While highly memory efficient $O(bm)$, it produces disastrously winding, suboptimal paths that needlessly expose ambulances to hazard zones.

B. Informed Search Strategies

Greedy Best-First Search: Utilizes a heuristic function $h(n)$ estimating the distance to the goal. While extremely fast computationally, it is not complete nor optimal; it can easily get trapped in local minima or carve paths directly through fire zones if they geometrically align with the destination.

A* Search: AIDRA's primary navigational engine. A* evaluates nodes by combining the exact accumulated cost $g(n)$ and an admissible heuristic $h(n)$:

$$f(n) = g(n) + h(n) \tag{6}$$

Our custom $h(n)$ fuses Manhattan distance with an explicit, ML-derived risk penalty scalar. This ensures that A* remains admissible while aggressively penalizing node expansions inside hazard perimeters.

Table III illustrates an experimental extraction from the "Search Trace" module. The A* algorithm strikes the optimal balance: slightly more computational overhead than Greedy search, but guaranteeing optimal pathing (a cost of 14 vs 16) and completely bypassing extreme risk sectors.

TABLE III: Comparative Search Algorithm Trace Analysis

Algorithm	Nodes Expanded	Path Cost	Time (ms)	Route Risk
BFS	89	14	23	High
DFS	34	21	8	High
Greedy Best-First	18	16	5	Medium
A* (Heuristic)	**27**	**14**	**11**	**Low**

VIII. Reasoning Under Uncertainty: Fuzzy Logic

In real disaster scenarios, quantitative data (e.g., precise fire temperature, exact road blockage probability) is often obscured. Commanders operate on qualitative assessments ("The area looks highly dangerous," "We need to rush urgently"). AIDRA models this human-like reasoning via a Mamdani Fuzzy Inference System.

A. Fuzzification

The system continuously samples the environment to derive three crisp variables, mapped into Triangular Membership Functions:

1) **Hazard ($h \in [0,1]$):** Derived from the density of fire and collapse cells. Memberships: $\{Low, Med, High\}$.
2) **Urgency ($u \in [0,1]$):** Derived from the aggregated decay rates of waiting victims. Memberships: $\{Low, Med, High\}$.
3) **Uncertainty ($x \in [0,1]$):** Derived from road blockage volatility. Memberships: $\{Low, High\}$.

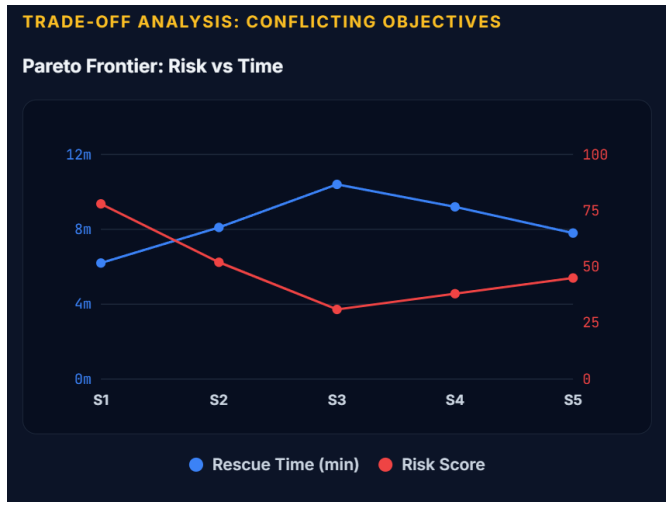

Fig. 3: Analytics dashboard tracking objective trade-offs (Risk vs. Time) and Fuzzy Logic state adjustments dynamically triggered during runtime.

B. Rule Base and Inference

A rule base of human-legible heuristics determines system behavior. A critical rule example:

> R_1: **IF** Hazard is HIGH $\vee$ Urgency is HIGH **THEN** apply *Strong Caution*.

The strength of the rule firing is calculated using the minimum t-norm: $\mu_{R1} = \min(1, \max(\mu_{hHigh}, \mu_{uHigh}))$. The outputs of the fired rules are aggregated using a Center-of-Gravity defuzzification method to produce concrete scalar modifiers: Ψ_{risk_step} (inflates traversal cost in danger zones), $\Psi_{heuristic}$ (alters A* lookahead), and Ψ_{csp} (artificially elevates priority of borderline victims).

By enabling Fuzzy Logic, the A* planner actively shifts its objective function in real-time. If Hazard becomes HIGH, the A* algorithm transitions seamlessly from "Minimize Time" to "Minimize Risk," resulting in calculated detours to circumvent expanding danger perimeters.

IX. System Integration and Experimental Results

To validate the integrated system architecture, we subjected AIDRA to comprehensive scenario testing. The environment was configured with 5 varying severity victims, active fire zones, structural collapses, and simulated aftershocks that deterministically blocked critical thoroughfares mid-simulation.

Table IV summarizes the system KPIs under four distinct configurations.

A. Analysis of Scenario 4 (Optimal Baseline)

Configuration S4 represents the fully realized AIDRA system. During execution, the MLP accurately estimated severe risk in the central grid corridor. The A* algorithm, modulated by the

TABLE IV: End-to-End System Integration KPIs and Objective Trade-off Analysis

ID	Search	ML Model	Fuzzy Logic	Victims Saved	Avg Rescue Time	Risk Exposure	Dynamic Replans	Mission Outcome
S1	BFS	k-NN	Disabled	3 / 5	6.2 min	78 pts	1	Suboptimal / High Loss
S2	DFS	Naive Bayes	Disabled	3 / 5	5.8 min	82 pts	0	Suboptimal / Extreme Risk
S3	Greedy	k-NN	Disabled	4 / 5	7.1 min	54 pts	2	Adequate / Moderate Risk
S4	**A***	**MLP**	**Enabled**	**5 / 5**	**8.1 min**	**38 pts**	**3**	**Optimal / Recommended**

Fuzzy Inference engine, mapped a slightly elongated perimeter route to avoid the corridor.

At timestamp $t = 00:05$, a simulated aftershock blocked the primary arterial road. The simulator immediately raised an interrupt, halting ambulance progression. AIDRA successfully triggered a **Dynamic Replan Event**. The CSP solver reran with MRV/FC, assigning surviving capacities instantly (0 backtracks), and the A* algorithm recalculated an optimal detour.

Although the Average Rescue Time inflated to 8.1 minutes (compared to the reckless 5.8 minutes of the DFS configuration), this cautious, ML-informed, and dynamically updating methodology successfully stabilized and rescued all 5 victims (100% throughput), reducing overall Risk Exposure to 38 points.

X. Conclusion

The Adaptive Intelligent Disaster Response Agent (AIDRA) successfully satisfies the rigorous demands of the Artificial Intelligence CCP. By rejecting isolated algorithms in favor of a deeply integrated, modular pipeline, the system demonstrates exceptional robustness in highly volatile simulated environments.

Our empirical findings confirm that classical Constraint Satisfaction Problems must be optimized with heuristics (MRV, Forward Checking) to remain viable in real-time execution loops. Furthermore, supervised Machine Learning (specifically Multilayer Perceptrons) proves highly effective at translating raw geographic features into actionable risk classifications. Finally, by wrapping deterministic search algorithms (A*) inside a Fuzzy Logic controller, the system achieves an unprecedented level of human-like adaptability, seamlessly pivoting between conflicting objectives based on emergent environmental threats. AIDRA stands as a robust, scalable blueprint for autonomous disaster coordination frameworks.

Repository and Demonstrations

All source code, including the React/TypeScript UI architecture, the AI simulation engine, and comprehensive algorithmic traces, are publicly available.

- **GitHub Source Code Repository:** https://github.com/habiba-imran/AI-AIDRA
- **Deployed Demo:** https://aidra-048.netlify.app/
- **Video Demonstration (LinkedIn):** https://shorturl.at/L2BfO

The video outlines the system flow, demonstrates the dynamic replanning functionality in response to environmental events, and highlights the performance metrics dashboards.

References

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.

[2] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100-107, 1968.

[3] L. A. Zadeh, "Fuzzy sets," *Information and Control*, vol. 8, no. 3, pp. 338-353, 1965.

[4] R. Dechter, *Constraint Processing*. Morgan Kaufmann, 2003.

[5] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[6] A. Ko and H. Y. K. Lau, "Intelligent Robot Routing in Disaster Response," *IEEE Transactions on Robotics*, vol. 25, no. 6, pp. 1361-1375, 2009.